Calibrate, Then Accumulate

A 39th-place solution to the ADIA Lab Structural Break Challenge: Real-Time Edition

cybergeek
Independent developer and researcher
Model: `worthwhile-raven` · Final submission: 30 · October 2026

Abstract

Our solution combines a history-calibrated statistical representation with gradient-boosted trees and small causal neural models. For each series, we fit a model of normal behaviour to its provided history, transform incoming observations into several residual streams, and compute multiscale change statistics. Trees trained on different augmentation distributions supply the main detector. A causal patch TCN and a neural Shiryaev–Roberts-style accumulator provide complementary temporal evidence; a small collection of fixed blends and difference corrections completes the submitted ensemble.

The selected submission scored **65.34% public TS-AUC** and **65.2271% private TS-AUC**, placing **39th in the official team-aware ranking**. The public score is not entirely out of sample: the final training recipe includes the 100 labelled reduced-test series supplied by the organisers, which also belong to the public leaderboard. Their contribution to the full public score has not been isolated. We do not interpret the public improvement from adding them as evidence of private-test improvement.

The main lesson was not that a larger ensemble always helps. Most of our progress came from making heterogeneous series comparable, improving training coverage, and combining genuinely complementary scoring functions. Late, small refinements were much harder to validate than their apparent complexity might suggest.

1. The task and the evaluation target

Each example contains a historical segment known to precede any labelled break, followed by an online segment revealed one observation at a time. At online step `t`, the output is a score for the event **"a structural break has already occurred by t"**. The break location is unknown. This is not the earlier competition's task of classifying a supplied boundary using an entire post-boundary segment.

The provided training population contains 10,000 series and 5,036,517 online observations. In the inspected training data, histories contain approximately 1,000–5,000 points and online segments 10–999 points. About half the series have a break. Each online prefix must be processed without inspecting future values, the eventual online length, or other test series.

TS-AUC measures ranking **across series at the same online step**, not an AUC computed independently inside each series. If `P_t` and `N_t` are the positive and negative counts at step `t`, the metric is:

```
TS-AUC = sum_t P_t * N_t * AUC_t / sum_t P_t * N_t
```

Ties receive half credit. Consequently, neither next-point forecasting loss, a stopping-time objective, nor ordinary pooled row-wise AUC is an adequate substitute. We used exact per-step, pair-count-weighted evaluation and checked key results against an independent sklearn implementation.

2. Start with a normal reference for each series

The training series are heterogeneous. A level, a variance, or an autocorrelation value that is unusual for one process can be ordinary for another. Our representation therefore asks: **how does the observed prefix differ from this series' own historical behaviour?**

2.1 Three history-calibrated residual streams

For the main representation, an autoregression with an intercept and order up to 12 is fitted on the history, with BIC selecting the order. Its coefficients and historical innovation scale are then fixed during online inference. Let `e_t` be the innovation obtained by applying that predictor to the new observation and already-seen lags.

We construct:

- **z:** the AR innovation divided by its historical scale;
- **n:** the historical empirical probability-integral transform of the innovation, followed by a standard-normal quantile transform;
- **g:** the innovation additionally standardised by a causal EWMA variance estimate, with the smoothing choice fitted from the history.

Only the `n` stream is explicitly normal-score transformed. Standardising `z` or `g` does **not** guarantee that either is Gaussian, independent, or perfectly calibrated under the null.

A second whitening view uses longer AR candidates, with **AICc-weighted coefficient averaging** rather than a single long-order selection. This view is used by the longAR tree group. The BIC and longAR feature matrices must not be interchanged at inference.

2.2 Multiscale evidence, not a single alarm statistic

Across the three streams we use 26 channels: level, square, absolute value, sign, tail indicators, lag products up to lag 8, and ARCH-like dependence features. For each channel, a current window average is compared with the historical channel mean and scaled by its historical standard deviation and the square root of the available window size.

The time scales are 15 approximately square-root-of-two-spaced finite windows:

```
5, 7, 10, 14, 20, 28, 40, 57, 80, 113, 160, 226, 320, 453, 640
```

A sixteenth window covers everything observed since the online segment began. Scan summaries include maxima, the maximising scale, positive/negative CUSUMs and a Bayes-inspired aggregation across scales. Historical process descriptors and the known current time are also supplied. The resulting 583 base features are expanded by 180 derived interactions to **763 columns**.

These are z-style evidence features, not calibrated hypothesis-test p-values. Their usefulness comes from a learned combination of many imperfect views of change, rather than a claim that one hand-written detector captures every break type.

3. Tree ensembles: vary the training task, not just the seed

The main tree score averages four LightGBM groups, each with three seeds:

Group	Distinguishing training/representation choice
sim4	Additional simulated-break examples
shift	Move a tail of the positive series' history into the online segment to create an earlier monitoring start
advance	Move an initial online prefix into the history and shift the remaining break location
longAR	Longer, AICc-averaged historical whitening

All groups retain real training observations. The important diversity is the training distribution and whitening view, not simply a collection of independent seeds of one dataset.

3.1 Cover all real online steps while controlling their loss mass

An effective change was to use every real online step rather than a sparse selection. However, giving all those rows unit weight would also change the balance between real and augmented data. We retained the original effective mass by assigning real rows weight approximately **0.14994**, while ordinary augmented rows have weight 1. Transplant-side training uses three donor examples with weight 1/3 each rather than tripling that augmentation's mass.

This separates two interventions that are otherwise easy to confuse: increasing coverage and changing source weights. It controls relative source loss mass, not within-series dependence; validation must therefore remain grouped by parent series.

The final base boosters use conservative settings, including learning rate 0.02, 31 leaves, large minimum leaf support and L2 regularisation. Most have 400 boosting rounds. Two sim4 seeds use 1,200 rounds; the third retains 400.

3.2 Real-change transplants as side models

We also trained six side boosters: three each for the shift and longAR groups. They use real post-break donor material transplanted into other series, rather than relying exclusively on parametric simulated breaks. The final side boosters have 1,200 rounds.

At inference, each affected group is a **25% original / 75% transplant-side blend**. The four group scores are then averaged equally. The placement matters: making a donor-trained model stronger in isolation did not necessarily improve an ensemble when it replaced or homogenised several existing members.

For cross-validation, augmentations and donor construction must follow the training parents of the outer fold. Generating augmented data from the full population before splitting would invalidate the comparison.

4. A complementary neural view

The neural members see history-standardised raw values together with the `z`, `n` and `g` streams. They use causal patch representations and running statistics; they are not unrelated pretrained foundation models.

4.1 Causal patch TCN

The TCN compares the current, partially observed patch with historical and earlier online patch representations, together with explicit streaming statistics. A point in an unfinished patch must not be represented using the later points in that patch.

The submitted TCN averages two seeds. Training uses real examples plus start-advance augmentation, history-patch sampling, sign reflection, and pair-weighted logistic supervision at sampled online anchor times. The standard recipe uses 60 epochs and AdamW with a warm-up/cosine learning-rate schedule.

Its value was complementarity. A neural member did not need to beat the tree ensemble alone to improve the combined ranking. In the historical comparison, start-advance training reduced the pooled OOF logit correlation with the tree baseline from about 0.920 to 0.854.

4.2 Neural evidence accumulation

A second member learns per-point evidence increments and accumulates them in log space:

```
r_t = softplus(r_(t-1)) + increment_t
score_t = sigmoid(r_t + bias + training_time_prior_t)
```

Initialise `R_0 = 0` (equivalently, `r_0 = −∞`), so the first update is `r_1 = increment_1`.

Equivalently, before the final readout, this has the Shiryaev–Roberts form `R_t = (1 + R_(t-1)) * exp(increment_t)`, collecting support over possible starts. The increments are learned discriminatively. We do **not** claim that they are true calibrated log-likelihood ratios, or that the final ensemble is an exact Bayesian posterior.

The final CUSUM-style member averages eight seeds. It uses real examples, two start-advance copies and shift augmentation. An early/late hand-over combines the two neural families:

```
alpha_t = clip((t - 32) / 96, 0, 1)
N_t     = (1 - alpha_t) * TCN2_t + alpha_t * CUSUM8_t
B_t     = 0.70 * Tree_t + 0.30 * N_t
```

The ramp is zero through step 32 and one from step 128 onward. It lets direct patch evidence dominate early and accumulated evidence dominate later.

5. What the final 53-model submission actually computes

The remainder of the final solution consists of small blends and difference corrections. It should not be represented as 53 models all averaged with equal weights.

Component	Fitted models	Role
Four base tree groups	12	Main multiscale detector
Transplant side boosters	6	Within-group side blends
Causal TCN	2	Early neural evidence
Neural CUSUM	8	Accumulated neural evidence
Reset evidence networks	2	Complementary probability blend
Prefix correction	1	History-routed replacement difference
Paired comparator	2	Small two-segment comparison blend
Signature and moments models	4	Matched prediction difference
Direction models and priors	16	History-informed direction difference
Total	**53**	**38 tree boosters and 15 neural models**

The Reset member uses a GRU-style gated transformation whose hidden state is reset to a history-derived state at every online step. Temporal information still enters through running statistics and evidence accumulation; it is not a detector with no memory of the prefix.

The prefix model adds a small difference relative to one Reset seed. It reads full-prefix evidence with an age-grid readout, not a separate suffix model for every candidate start. Its weight is higher for steps 257–512. A history-only predictability measure, `abs(log(sigma_AR / std(history)))`, routes the correction with multiplier 1.5 below 0.0137378481 and 0.5 otherwise.

The comparator supplies another two-segment view. The signature term is the difference between path-signature and matched moments models, rather than an unqualified claim that signatures alone are superior. Direction priors estimated from history modulate signed change evidence; a matched base prediction is subtracted before applying the correction.

Writing `clip` for clipping to `[0,1]`, the final order is:

```
B = 0.70 * Tree + 0.30 * Neural
U = clip(0.90 * B + 0.10 * Reset2
         + route(H) * w_t * (Prefix - Reset937))
V = clip(0.95 * U + 0.05 * Comparator2
         + 0.05 * (Signature2 - Moments2))
S = clip(V + 0.1108333333 * mean(Direction - MatchedBase))
```

The signature difference starts at step 32. `w_t` is 0.15 at steps 257–512 and 0.05 otherwise. Clipping occurs after the prefix, comparator/signature and direction stages—not separately after every individual operation.

These additions explain the final implementation, but their presence does not mean each has equally strong independent evidence. Several late refinements were small and affected by repeated development on the same data.

6. Results: separate public, private and local evidence

6.1 Selected submission

Evaluation	TS-AUC	Interpretation
Selected submission, public	**65.34%**	Includes a labelled-public-series training component
Selected submission, private	**65.2271%**	Official final out-of-sample result
Official final ranking	**39th**	Team-aware ranking

The public-to-private change is about −0.11 percentage points. Because these are different test populations, we cannot attribute that difference exclusively to training on the 100 public series.

6.2 Development milestones

The following are public leaderboard readings, not a common-protocol ablation table. A later row may contain several changes and the intervening development was not monotonic.

Submission	Main stage reached	Public TS-AUC
1	History-calibrated residual statistics and LightGBM	63.00%
2	Finer window grid, interactions and seed averaging	63.48%
9	Full real-step coverage with controlled loss mass	63.99%
11	Causal TCN with start-advance training	64.53%
13	Neural accumulation and revised core blend	64.66%
17	Reset/prefix additions and transplant side models	65.01%
25	Later corrections, more CUSUM seeds and longer side boosters	65.23%
30	Retains submission 28's public100 refit; repairs training orchestration	65.34%

The early EWMA z-score baseline scored about 51.45% on the provided training population, without learned parameters. This is a local baseline reading, not a public score to subtract from 63.00% as a controlled public improvement.

7. Public-label and data-use disclosure

The organisers supplied 100 labelled reduced-test series, confirmed to belong to the public leaderboard and not the private test set. Three advance boosters had already used these as ordinary training examples; the final refit included them across the remaining model families. In total, 50 fitted model files were retrained and three advance files retained, with the history-routing threshold recomputed.

This can raise the public score through an in-sample component. We did not isolate its contribution to the complete public metric, so we do not count that gain as generalisation evidence.

A separate research probe, submission 16, explicitly used the known public labels and reached about 65.70% publicly. **It was not selected for final evaluation and is not the final model described here.** The selected model uses the public series as training inputs, not a label-lookup rule at inference.

The final reconstruction recipe uses the official training files and the labelled reduced-test files. It does not include hidden test series as training inputs or externally pretrained time-series weights. The submitted repaired package does not bundle the competition datasets.

We did not add an explicit float-grid artifact detector or artifact-specific decision rule. This is not a claim that learned models are automatically insensitive to every data-generation artifact, or that we have quantified artifacts' effect on the final ranking.

8. What helped, and what we would do differently

History calibration was foundational. It made a heterogeneous population amenable to shared learning. Explicit multiscale features were more useful than relying on a global anomaly threshold.

Training distribution mattered as much as model family. Shifted starts, simulated breaks and donor transplants gave the same learner different errors. More samples or a stronger individual model did not guarantee a stronger ensemble.

Complementarity had to be measured in the full recipe. Direct TCN scoring, accumulated neural evidence and the Reset member could help despite imperfect standalone scores. Low correlation alone was not sufficient: weak alternatives often still hurt.

Normal prediction quality and break detection are different objectives. In one controlled conditional-density experiment, normal next-point NLL improved by roughly 0.019–0.023 nat, but a fixed-window NLL detector failed to add useful break information. That is a negative result for that construction, not an upper bound on learned likelihood-based detection.

Late refinement had diminishing practical returns. Tested configurations of larger/alternative neural models, external time-series pretraining, distillation, learned stacking, additional statistical feature families and specialist routing often failed to deliver reliable gains over the ensemble of the time. We would not generalise those results to entire method families.

In retrospect, we would establish a verified probabilistic baseline earlier, preserve all OOF predictions separately from disposable caches, and spend less effort on many small corrections before securing cleaner confirmation. AI-assisted implementation accelerated the research, but also made explicit protocol checks, independent scoring and careful interpretation essential.

9. Engineering, causality and reproducibility

Inference is CPU-based and requires NumPy, SciPy and LightGBM, with an optional compiled C++ signature backend and a NumPy fallback. PyTorch is used for training, not required for the final pretrained inference path. Each series has its own inference state; no test-data adaptation across series or batch-level rank normalisation is part of the submitted recipe.

The selected cloud run completed on CPU in approximately **2 h 31 min**. The official local determinism check passed at **1e−8 on the pretrained path**; we also performed raw-input replay and prefix/ordering checks. This is not a claim of bitwise-identical retraining across GPUs, library versions or machines.

Submission 30 retains submission 28's fitted weights, ensemble configuration and inference code. The engineering repair changed training orchestration, input resolution and provenance, while removing bundled competition data. A low-budget raw-input smoke reconstruction exercised all model families. A full-budget, end-to-end scratch retraining of the repaired package was **not** repeated as part of that repair; we distinguish that limitation from unchanged pretrained inference.

This document describes the submitted recipe and its evidence. It does not redistribute the competition data or claim that source code or weights have already been publicly released.

Conclusion

The useful abstraction was **calibrated evidence about change**, not a universal anomaly score. Historical modelling, multiple time scales, diverse training tasks and learned accumulation together produced a competitive causal detector. The final private result was 65.2271% TS-AUC and 39th place.

The weaker part of the process was inference about research progress: public-score improvements, local component gains and complete-ensemble generalisation were sometimes too easy to conflate. A successful write-up should preserve those distinctions as carefully as the implementation preserves causality.

References and scope

- Official Real-Time Edition task and evaluation documentation
- Official out-of-sample leaderboard
- ADIA Lab competition page
- CrunchDAO guidance on write-ups

Numerical milestones describe the historical experiments indicated, not interchangeable estimates of one model.